SNOWFLAKE DATA ENGINEERING PROJECT

Healthcare Intelligence Hub

Patient, Cost & Care Analytics

Phase 8 — Project Demonstration

Institutional template-based phase document

Student	Aashi Bindal
Enrollment No.	0827CI241004
Team ID	[Team ID]
Institution	Acropolis Institute of Technology and Research · Department of CSIT
Faculty Guide	[Faculty Guide]
Academic Session	[Academic Session]
Document date	October 2026

Prepared from the verified project source (query.sql, Health/streamlit_app.py, Health/snowflake.yml, six CSV datasets). Bracketed fields are placeholders for information not supplied.

Contents

8.1 Presentation-Ready Overview 1

8.2 Demonstration of Proposed Features 1

8.3 Team Involvement in Demonstration 2

8.4 Project Demo Planning 3

8.5 Communication 4

8.6 Step-by-Step Setup and Execution (for the demo machine) 5

8.7 Demonstration Script (≈ 9 minutes) 5

8.8 Suggested Screenshots to Capture 6

8.9 Frequently Asked Technical Questions 6

8.10 Known Limitations 7

8.11 Scalability & Future Plan 7

Phase 8 — Project Demonstration

Project: Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Templates covered:** Demonstration of Proposed Features · Team Involvement in Demonstration · Project Demo Planning · Communication · Scalability & Future Plan (1 mark each)

> **Availability note.** The files show that the Snowflake objects and the Streamlit app were written, and `snowflake.yml` holds an auto-generated app identifier. A live Snowflake account, deployed app or demo recording could **not** be verified. Expected on-screen values below come from the local replica (Phase 6). Confirm them in your own account before presenting. Team-specific fields are placeholders.

8.1 Presentation-Ready Overview

Problem	Hospital administrators and finance teams need a consolidated, queryable view of patient visits — spanning cost, length of stay, satisfaction and readmissions — that today sits fragmented across raw operational records. Without a structured warehouse, it is difficult to answer basic questions such as which facilities and doctors are performing well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk.
Solution	Snowflake pipeline `RAW → STAGING → CURATED → MART` plus a Streamlit-in-Snowflake dashboard with 6 KPIs, 8 charts, 2 tables and 5 filters.
Data	6 CSVs, 14,059 rows; 10,000 visits; January 2024 to August 2026.
Stack	Snowflake SQL, internal stage and file format, Streamlit in Snowflake, Python 3.11, pandas, Altair, Snowpark.
Evidence	55 automated checks on a local replica: 50 pass, 5 findings documented with fixes.

8.2 Demonstration of Proposed Features

Field	Value
Date	15 March 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

This document lists every feature proposed during planning and tracks whether each was implemented and demonstrated.

Table 8.1 — Proposed features and status (implementation status verified against code)

S.No	Feature Name	Description	Status (Implemented / Partial / Pending)	Demonstrated (Yes / No)	Remarks
1	Layered Snowflake database	`HEALTH_DB` with RAW, STAGING, CURATED, MART	Implemented	[Yes/No]	`query.sql`

S.No	Feature Name	Description	Status (Implemented / Partial / Pending)	Demonstrated (Yes / No)	Remarks
2	Ingestion objects	`CSV_FORMAT` , `HEALTHCARE_STAGE` , 6 typed RAW tables	Implemented	[Yes/No]	—
3	Data load to RAW	Load 6 CSVs into RAW	Partial	[Yes/No]	No `COPY INTO` in repo; UI load assumed
4	Cleaning & standardisation	TRIM, INITCAP, NOT NULL keys	Implemented	[Yes/No]	`ENT` → `Ent` side effect
5	Star schema	5 dimensions + visit fact	Implemented	[Yes/No]	`DIM_DATE` not joined
6	Data-quality checks	Row counts, NULLs, duplicates, orphans	Implemented	[Yes/No]	Manual review of results
7	KPI views	10 MART views	Implemented	[Yes/No]	Doctor-view grain defect D-01
8	KPI cards	6 metrics	Implemented	[Yes/No]	—
9	Trend & mix charts	Monthly trend, visit type, top diagnoses, admission, treatment	Implemented	[Yes/No]	—
10	Facility & doctor comparison	Facility chart and table; doctor leaderboard	Implemented	[Yes/No]	—
11	Readmission & cost analysis	Readmission by diagnosis; cost breakdown by visit type	Implemented	[Yes/No]	—
12	Interactive filters	Date, facility, specialty, visit type, diagnosis category	Implemented	[Yes/No]	—
13	Insurance / payer analysis	Coverage versus patient payment by insurer	Partial	[Yes/No]	Totals only (`VW_EXECUTIVE_KPI`)
14	Security & automation	RBAC, masking, scheduled refresh	Pending	No	Future scope

Feature Implementation Summary:

Metric	Value
Total Features Proposed	14
Total Features Implemented	11 fully + 2 partially
Total Features Demonstrated	[count after demo]
Overall Implementation Rate (%)	78.6% fully implemented (11 / 14); 85.7% counting partial features as half (12 / 14)

8.3 Team Involvement in Demonstration

Field	Value
Date	[Demo Date] *(template shows 15 March 2024)*

Field	Value
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

This document records each team member's participation and role during the demonstration. *(Roles are suggestions mapped to the demo plan; fill in actual names and participation.)*

S.No	Team Member Name	Role in Demo	Section Presented	Contribution Summary	Participation (Active / Passive)
1	[Name]	Lead presenter	Introduction & problem	Framing, objectives, wrap-up	[Active/Passive]
2	[Name]	Data engineer	Architecture & pipeline	Layers, SQL objects, data load	[ ]
3	[Name]	Analytics engineer	Data model & KPIs	Star schema, MART views, validation	[ ]
4	[Name]	App developer	Live dashboard	Filters and charts walkthrough	[ ]
5	[Name]	QA	Testing results	Test summary and defects	[ ]
6	[Name]	Q&A coordinator	Q&A	Routes questions	[ ]

Aspect	Details
Team Leader / Coordinator	[Name]
Overall Team Coordination Rating (1–5)	[ ]
Any issues during demo	[record, e.g., warehouse resume delay]
How issues were resolved	[record]

8.4 Project Demo Planning

Field	Value
Date	15 March 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

Table 8.2 — Demo plan (10 minutes)

S.No	Demo Section	Description	Duration (mins)	Responsible Member
1	Introduction & problem	Persona, pain points, objectives	1.0	[Member 1]

S.No	Demo Section	Description	Duration (mins)	Responsible Member
2	Architecture & data model	Architecture diagram; layers; star schema	1.5	[Member 2]
3	Pipeline in Snowflake	Show schemas and objects; run row-count and orphan checks	2.0	[Member 2/3]
4	KPI views	Query `VW_EXECUTIVE_KPI` and `VW_READMISSION_ANALYTICS`	1.0	[Member 3]
5	Live dashboard	KPIs, filters, charts, leaderboard	3.0	[Member 4]
6	Testing, limitations & future	Test summary, defects, roadmap	1.5	[Member 5]

Demo Flow Summary:

Step	Activity	Notes
1	Introduction & Problem Statement	One slide: the official problem statement and its three questions
2	Solution Overview	Architecture figure (Phase 3, Figure 3.1)
3	Live Feature Demonstration	Snowsight worksheet → Streamlit app (script in §8.7)
4	Q&A Session	Use FAQ §8.9; keep this documentation open

8.5 Communication

Field	Value
Date	15 March 2026 *(as printed on template)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

The project files contain no record of team communication. The channels below are a plan; confirm or edit them to match what the team actually used.

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
1	Team Standup	Daily	[e.g., WhatsApp / Google Meet]	All members	Yesterday / today / blockers
2	Progress Update	Weekly	[e.g., Google Meet + shared Drive]	Team + mentor	Sprint progress, burndown
3	Issue / Bug Discussion	As Needed	[e.g., GitHub Issues / chat]	Relevant members	Resolve SQL or app defects
4	Stakeholder Review	Bi-Weekly	[e.g., Meet with faculty guide]	Team + [Faculty Guide]	Validate scope and deliverables
5	Final Demo Rehearsal	Once	[In person / Meet]	All members	Timing, hand-offs, fallback plan

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
6	Documentation review	Before submission	Shared Drive / GitHub PR	All members	Fill placeholders, proof-read

S.No	Challenge Faced	Resolution / Action Taken
1	[e.g., differing local setups]	[e.g., used Snowsight only]
2	[ ]	[ ]
3	[ ]	[ ]

8.6 Step-by-Step Setup and Execution (for the demo machine)

1. Log in to Snowsight with a role that owns `HEALTH_DB` (or can create it). Select warehouse `COMPUTE_WH` and run `ALTER WAREHOUSE COMPUTE_WH RESUME IF SUSPENDED;` a minute before the demo to avoid a resume delay.
2. If the database does not exist yet: follow Phase 5 §5.1.2 (create objects → upload CSVs → load → run the rest of `query.sql`).
3. Open the Streamlit app (Projects → Streamlit → "HEALTH") and wait for the first load (cache is cold).
4. Keep a worksheet open with the demo queries in §8.7.
5. Fallback: if Snowflake is unreachable, present the figures in Phase 5 and run `python tests/local_validation.py` locally to show the pipeline logic.

8.7 Demonstration Script (≈ 9 minutes)

0:00 – Introduction (1 min). "Hospital administrators and finance teams need one consolidated, queryable view of patient visits, covering cost, length of stay, satisfaction and readmissions. Today that data sits fragmented across raw operational records. Without a structured warehouse they cannot easily see which facilities and doctors perform well, which diagnoses drive the longest and most expensive stays, or where satisfaction is at risk. We built that warehouse in Snowflake, with a native Streamlit dashboard on top."

1:00 – Architecture (1.5 min). Show Figure 3.1. "CSV files land in an internal stage and load into typed RAW tables. STAGING trims and standardises text and drops rows with missing keys. CURATED holds a star schema with a 10,000-row visit fact and five dimensions. MART holds ten KPI views. The app reads the curated fact through the active Snowpark session, so there are no credentials in code."

2:30 – Pipeline in Snowflake (2 min). In the object explorer, expand `HEALTH_DB` to show the four schemas. Run:

```
SELECT 'FACT_PATIENT_VISITS' AS T, COUNT(*) FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
UNION ALL SELECT 'DIM_PATIENT', COUNT(*) FROM HEALTH_DB.CURATED.DIM_PATIENT;
-- expected (local replica): 10000, 3000

SELECT COUNT(*) AS ORPHANS FROM HEALTH_DB.CURATED.FACT_PATIENT_VISITS
WHERE FIRST_NAME IS NULL OR DOCTOR_NAME IS NULL OR FACILITY_NAME IS NULL OR DIAGNOSIS_NAME IS NULL;
-- expected: 0
```

4:30 – KPI views (1 min).

```
SELECT * FROM HEALTH_DB.MART.VW_EXECUTIVE_KPI;
-- expected: 10000 | 2889 | 60 | 10 | 82818797.00 | 59918913.84 | 22899883.16 | 3.83 | 0.97 | 1203

SELECT DIAGNOSIS_NAME, READMISSION_RATE_PCT FROM HEALTH_DB.MART.VW_READMISSION_ANALYTICS LIMIT 3;
-- expected: Depression 14.05 | Back Pain 13.98 | Asthma 13.75
```

5:30 – Live dashboard (3 min). 1. Unfiltered KPI row: 10,000 visits; 2,889 patients; total cost; 3.83 / 5; 12.0%; 0.97 days. 2. Set **Visit Type = Inpatient**: visits fall to 1,261, LOS rises to 7.69 days, and room cost appears in the cost breakdown. "Room charges almost double inpatient cost." 3. Reset, then set **Diagnosis Category = Mental Health**: the readmission chart shows Depression on top. 4. Choose a **Facility**: the facility table narrows; point out average cost and satisfaction. 5. Scroll to the **Doctor Performance Leaderboard** and sort by satisfaction.

8:30 – Testing and roadmap (1.5 min). "We executed 55 automated checks on a replica of the pipeline. All structural checks pass. Five findings are documented with fixes: a doctor-grouping bug, two INITCAP label changes, and two source-data anomalies. Next steps: scripted loads with Tasks, RBAC and masking, and SQL push-down for scale."

8.8 Suggested Screenshots to Capture

#	Screen	Purpose / where it is used
S1	Snowsight object explorer showing `HEALTH_DB` and its 4 schemas	Phase 3/5 evidence
S2	Stage file list for `HEALTHCARE_STAGE`	Ingestion evidence
S3	RAW row-count query result	TC-SF-01
S4	STAGING NULL / duplicate check results	Phase 6
S5	Orphan check result (0)	Phase 6
S6	`VW_EXECUTIVE_KPI` result	Phase 7 Results
S7	App unfiltered: header, KPI row, sidebar	Phase 7 §7.1 placeholder
S8	App with Visit Type = Inpatient	Cost and LOS story
S9	Readmission chart with Mental Health filter	Quality story
S10	Facility performance chart and table	Facility comparison
S11	Doctor leaderboard	Doctor comparison
S12	`snowflake_validation.sql` outputs and Query History	Phase 6 Snowflake tests and performance

Save the files under `docs/screenshots/` with the names `S1_object_explorer.png` and so on.

8.9 Frequently Asked Technical Questions

Q1. Why Snowflake rather than MySQL? Storage and compute scale independently, the SQL is standard, the internal stage and file format handle CSV loading, and Streamlit runs natively, so the app needs no separate server or stored credentials.

Q2. Why four schemas? Separation of concerns and traceability. RAW keeps data as loaded, STAGING cleans it, CURATED models it for analysis, and MART holds consumption logic. Any layer can be rebuilt from the one before.

Q3. What is the grain of the fact table? One row per patient visit (`VISIT_ID`), 10,000 rows.

Q4. Why LEFT JOIN in the fact build? So that a visit is never lost when reference data is missing. The orphan check then counts any unmatched rows. There are 0 in this data.

Q5. Why is the fact table denormalised? The app and views can read one table with no joins. The trade-off is redundancy, and dimension changes require a rebuild.

Q6. Is DIM_DATE used? It is built but not yet joined. Monthly trends use `DATE_TRUNC` on `VISIT_DATE` . Joining it would enable weekday and weekend analysis.

Q7. How is the readmission rate calculated? `100 × count(READMISSION_WITHIN_30_DAYS = 'Yes') ÷ count(visits)` over the filtered set. The flag comes pre-computed in the source.

Q8. How does the app authenticate? `get_active_session()` returns the Snowpark session of the running Streamlit-in-Snowflake app. No passwords or keys appear in code.

Q9. Why does the app use pandas instead of the MART views? For simplicity: it loads the fact table once (cached for 600 s) and recomputes metrics for any filter combination in memory. For larger data, we would push filters and aggregations to SQL.

Q10. How did you test without Snowflake? `tests/local_validation.py` rebuilds the same SQL in SQLite, emulating Snowflake's `INITCAP` , and runs 55 checks. A Snowflake version of the checks is in `tests/snowflake_validation.sql` .

Q11. What defects did you find? Two doctors with identical name, specialty and department merge in the leaderboard. `INITCAP` changes `ENT` to `Ent` and `Follow-up` to `Follow-Up` . "Self Pay" visits show insurance coverage. Some admitted emergency visits have 0-day LOS.

Q12. Is patient data protected? The data appears synthetic. For real data we would add least-privilege roles, masking policies on names and date of birth, and remove names from the fact table.

Q13. How would you automate refresh? Load new files with Snowpipe or a scheduled `COPY INTO` , then rebuild STAGING and CURATED with a chain of Snowflake Tasks, or use Streams for incremental processing.

8.10 Known Limitations

Load not scripted · no RBAC or masking · app loads the full fact table into memory · KPI logic duplicated between views and app · `DIM_DATE` unused · INITCAP side effects · doctor-view grain defect · source anomalies (Self Pay coverage, zero-LOS admissions) · currency unspecified · Snowflake and UI tests not executed · synthetic data.

8.11 Scalability & Future Plan

Field	Value
Date	[Submission Date] *(template shows 15 March 2024)*
Team ID	[Team ID]
Project Name	Healthcare Intelligence Hub: Patient, Cost & Care Analytics
Maximum Marks	1 Mark

This section explains how the solution can scale to larger user bases, more data and more features, and gives the roadmap.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	Manual, unscripted data load and full rebuilds	Refresh depends on a person; not reproducible from the repository	High
2	No RBAC or masking; PII in the fact table	Unsuitable for real patient data	High
3	App loads the whole fact table into pandas	Memory and latency grow with volume	Medium
4	Doctor-view grain and INITCAP defects	Misleading leaderboard and labels	Medium
5	KPI logic duplicated in SQL and pandas	Risk of divergence	Low

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Single app on `COMPUTE_WH`; per-user cache	Multi-cluster warehouse (auto-scale) for concurrency; separate warehouse for ELT and BI
2	Data Storage	6 tables, ~14 K rows, full `CREATE OR REPLACE` each run	Incremental loads with Streams and `MERGE`; clustering on `VISIT_DATE` at large volumes; Time Travel for recovery
3	Performance	Full fact read, then pandas filters	Push filters into Snowpark or SQL (`WHERE` and `GROUP BY` in the warehouse); use MART views; dynamic tables for pre-aggregates
4	Security	Development role; no grants or masking	Role hierarchy (`HEALTH_ENGINEER` / `ANALYST` / `APP_VIEWER`), masking and row-access policies, app moved out of `RAW`

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Fix D-01 to D-03; script `COPY INTO`; run Snowflake and UI tests; RBAC and masking	[Next 2 weeks, estimate]	Correct, reproducible, secure baseline
Phase 3	Tasks or Snowpipe automation; Streams and `MERGE` incremental model; `DIM_DATE` weekday and seasonality views; insurer and payer dashboard page	[1–2 months, estimate]	Hands-off refresh; deeper finance insight
Phase 4	SCD Type 2 dimensions; Snowpark ML readmission-risk model; alerts when readmission rate exceeds a threshold; CI running the test scripts on every commit	[3+ months, estimate]	Predictive, monitored, production-grade analytics